

CS 423

Operating System Design:

Paging and TLBs

Sep 17

Ram Alagappan

Slide ack: Prof. Shivaram Venkataraman (Wisconsin)

Logistics

MP0: Will release grades soon

MP1: Due 09/30 mid-night

Use Piazza & TA office hours for MP help

Ok to post publicly if not including large amounts of your code

Or post privately, and let staff decide to make it public

AGENDA / LEARNING OUTCOMES

Memory virtualization

- Paging details

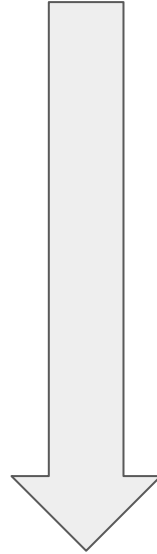
- Challenges with implementing paging

- Performance & TLBs

RECAP

Mechanisms for Virtualization

1. Time Sharing
2. Static Relocation
3. Base
4. Base+Bounds
5. Segmentation



Limited practicality, has many problems

More practical, still has some problems

PAGING

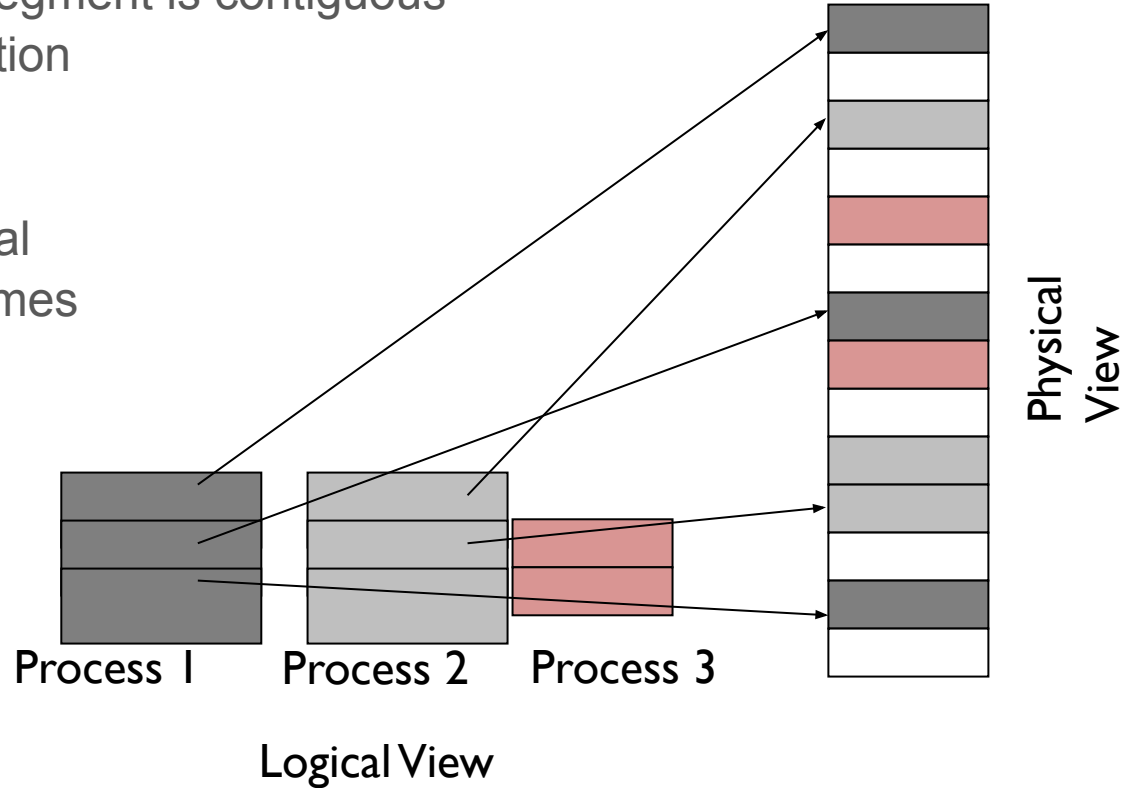
(most modern systems including Linux use this)

PAGING

Goal: Eliminate requirement that segment is contiguous
Eliminate external fragmentation

Idea:
Divide address spaces and physical
memory into fixed-sized pages/frames

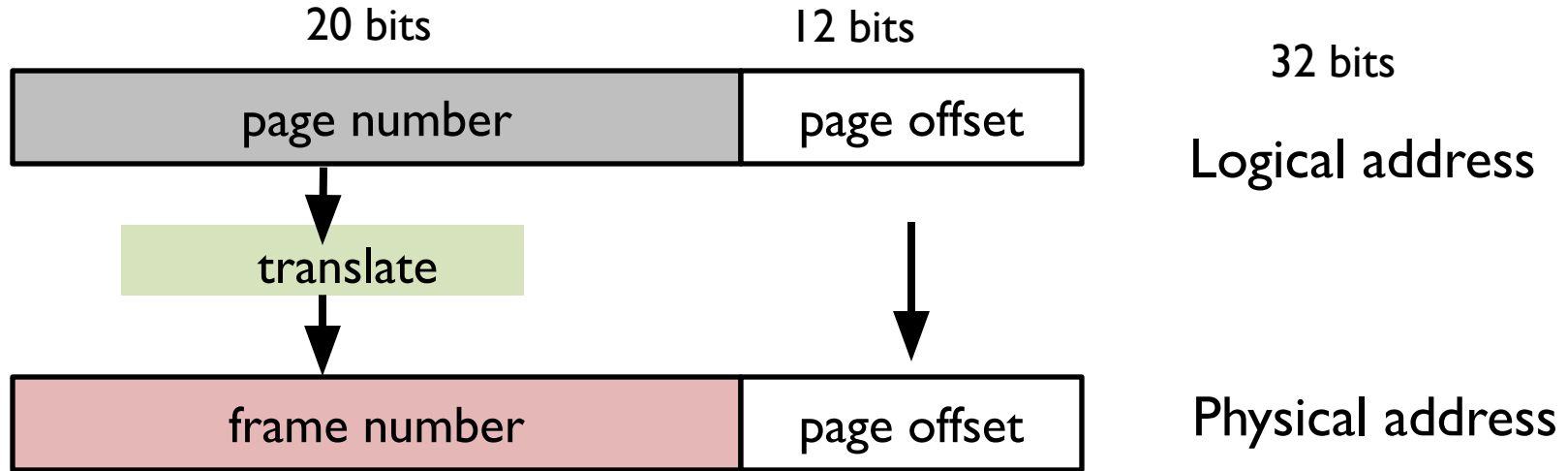
Size: 2^n , Example: 4KB



TRANSLATION of 4KB PAGE ADDRESS

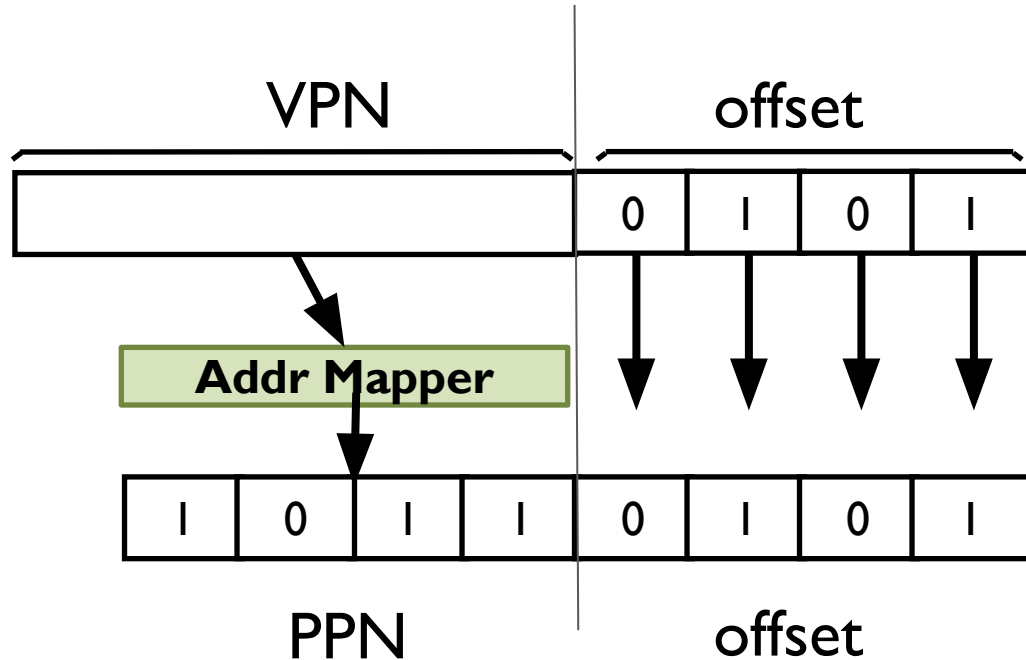
How to translate logical address to physical address?

- MSBs of virtual address => page number
- LSBs bits of virtual address => offset within page



VIRTUAL -> PHYSICAL PAGE MAPPING

#bits in virtual
address need not
be equal to #bits
in physical address



How should OS translate VPN to PPN/PFN?

VPN, PPN, Memory Sizes

What does $\#VPN \text{ bits} > \#PPN \text{ bits}$ mean?

Inverse?

Which case is typical?

What's the virtual address width of modern linux systems?

What's the physical address width of modern linux systems?

How much physical memory does the most powerful Linux server on AWS have? (U7i/U7inh)

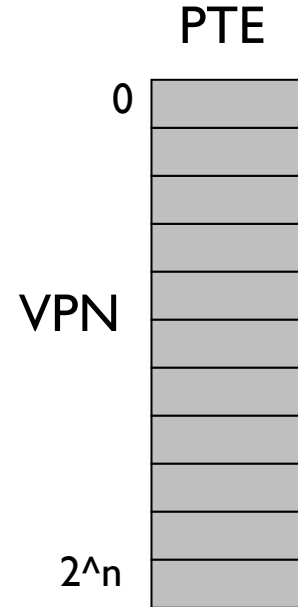
LINEAR PAGE TABLE

What is an obvious data structure?

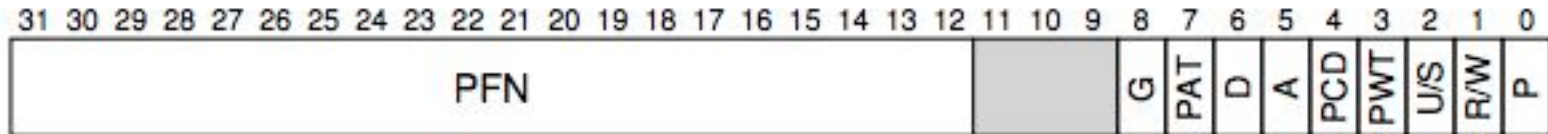
Linear page table (array)

Indexed by VPN

Yields PFN (Physical Frame Number)



A Single PTE (Page Table Entry):



HOW BIG IS A PAGE TABLE?

Assume 32-bit address

Assume 4KB pages => 12 bits for offset
20 bits for VPN

Assume 4 byte page table entries (PTE)

How large is PT for each process? $2^{20} * 4 = 4 \text{ MB}$

Implications? For 1K processes, we need 4GB of PTs!

END OF RECAP

WHERE TO STORE PAGE TABLES?

Store page table in memory (for now assume in physical memory)

Hardware stores page table base (for process) in special register

What is this register in x86?

On a context-switch:

Store contents of this register in PCB of old process

Load contents of this register with new process' page table base

HW could then use the table for translating VA to PA

MEMORY ACCESSES WITH PAGING

14 bit virtual address

```
0x0010: movl    0x1100, %edi
```

Assume PT is at phys addr 0x50000

Assume each PTE is 4 bytes

Assume 4KB pages

i.e., 12 bits for offset

How many memory accesses?

To which physical addresses?

Simplified view
of page table

2
0
...
...

MEMORY ACCESSES WITH PAGING

14 bit virtual address

```
0x0010: movl    0x1100, %edi
```

Assume PT is at phys addr 0x50000

Assume each PTE is 4 bytes

Assume 4KB pages

i.e., 12 bits for offset

Simplified view
of page table

2
0
...
...

Total: 4

2 PTEs + 2 (instr + data)

0x50000 for PTE of 0th page

load instr from 0x2010

0x50004 for PTE of 1st page

load from 0x0100 to %edi

Disadvantages of Paging

Additional memory reference to page table entry

- Extra memory access needed for each memory access!
- Inefficient, even if page table is stored in memory
- MMU stores only base address of page table
 - Solution: TLBs (today)

Space needed for page tables is too large

- Simple page table: requires PTE for each virtual page number
PTE needed even if page not allocated
- Page tables must be contiguously allocated
 - Solution: paging the page tables! (next)

Internal fragmentation: if process is 1K, with 4K pages, you waste 3K.
Why not makes pages really small? Tension?

PAGE TRANSLATION STEPS

For each mem reference:

1. extract **VPN** (virt page num) from **VA** (virt addr)
2. calculate addr of **PTE** (page table entry)
3. read **PTE** from memory
4. extract **PFN** (page frame num)
5. build **PA** (phys addr)
6. use **PA** to read from memory

Which steps are expensive?

EXAMPLE: ARRAY ITERATOR

```
int sum = 0;
for (i=0; i<N; i++){
    sum += a[i];
}
```

Assume 'a' starts at 0x3000
Ignore instruction fetches
and access to 'i' and 'sum'

Assume 4-byte PTE, what can you infer?
What is the PPN for VPN 3?
What is the PT base address?
Chat with neighbors for 2 mins...

What virtual addresses?

load 0x3000

load 0x3004

load 0x3008

load 0x300C

What physical addresses?

load 0x100C

load 0x7000

load 0x100C

load 0x7004

load 0x100C

load 0x7008

load 0x100C

load 0x700C

EXAMPLE: ARRAY ITERATOR

```
int sum = 0;
for (i=0; i<N; i++){
    sum += a[i];
}
```

Assume 'a' starts at 0x3000
Ignore instruction fetches
and access to 'i' and 'sum'

What virtual addresses?

load 0x3000

load 0x3004

load 0x3008

load 0x300C

What physical addresses?

load 0x100C

load 0x7000

load 0x100C

load 0x7004

load 0x100C

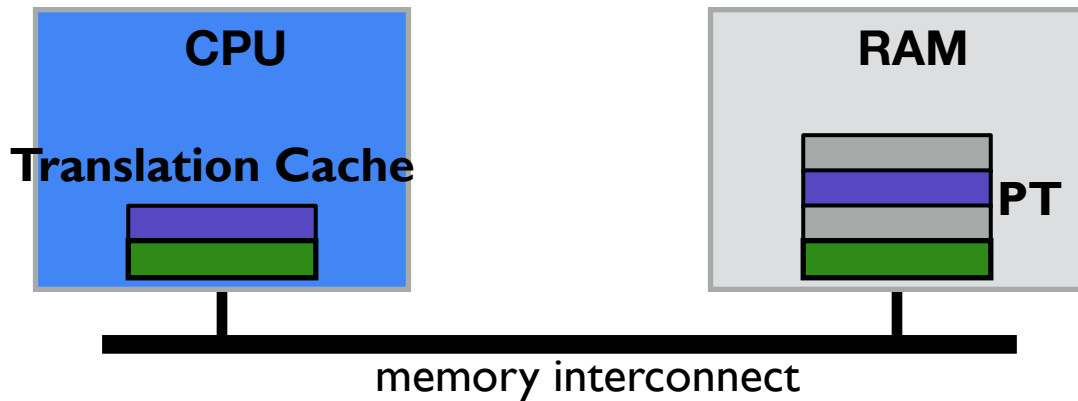
load 0x7008

load 0x100C

load 0x700C

Observation: repeatedly access the same PTE
because program accesses the same virtual page

STRATEGY: CACHE PAGE TRANSLATIONS



TLB: TRANSLATION LOOKASIDE BUFFER
Each core has its own TLB

TLB ORGANIZATION

TLB Entry

Tag (virtual page number)	Physical frame number (from PTE)	other bits
---------------------------	----------------------------------	------------

TLB housed in the MMU

Fully Associative

A given translation can be anywhere in the TLB

Hardware searches the entire TLB in parallel

Takes only 0.5 to 1 clock cycle to search

Typical size of TLB: 16 to 1024 entries (note: usually not bytes but #entries)

Other bits: valid, rwx (protection), dirty

ARRAY ITERATOR (W/ TLB)

```
int sum = 0;  
for (i=0; i<N; i++){  
    sum += a[i];  
}
```

Assume 'a' starts at 0x1000.
Ignore load & stores of 'i' and 'sum'

Assume following virtual address stream:

load 0x1000

load 0x1004

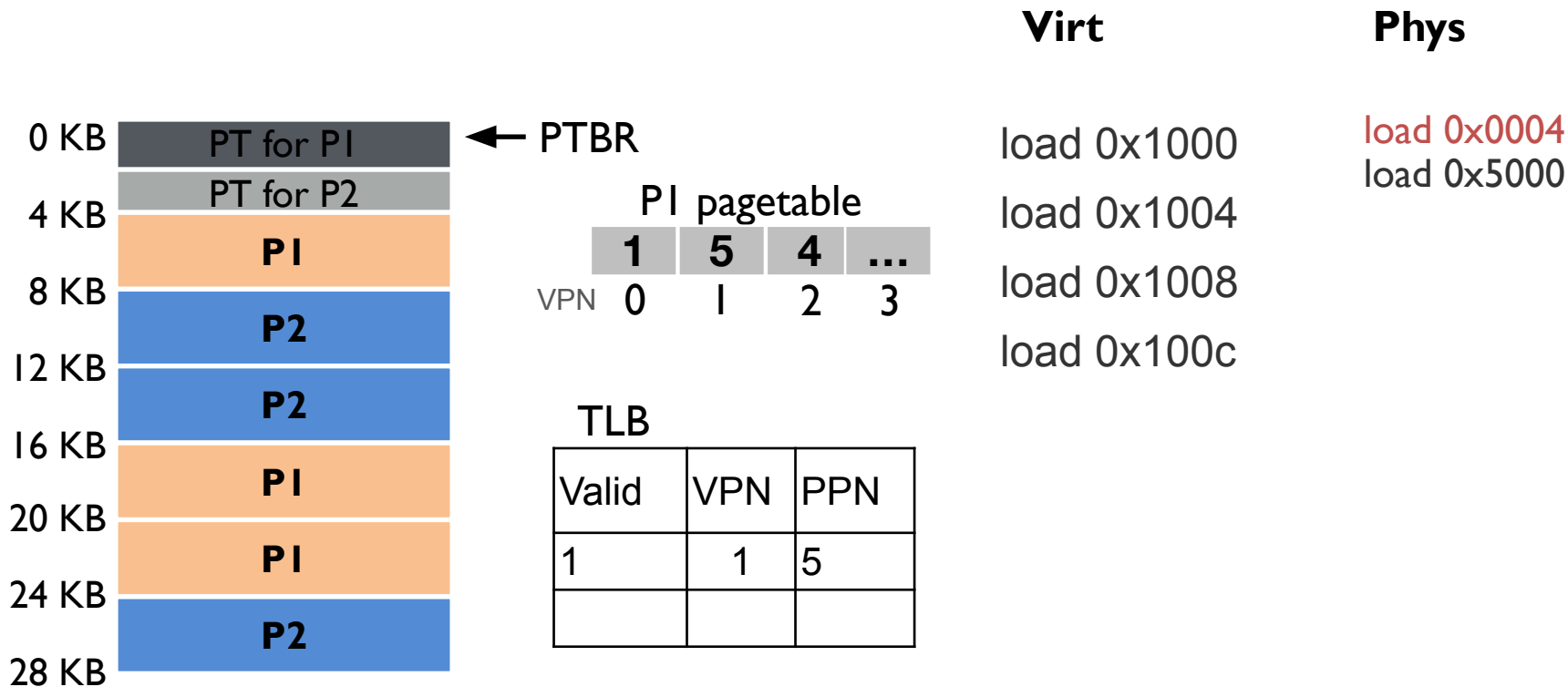
load 0x1008

load 0x100C

...

What will TLB behavior look like?

TLB Accesses: SEQUENTIAL Example



TLB Accesses: SEQUENTIAL Example



← PTBR

PI pagetable

	1	5	4	...
VPN	0	1	2	3

TLB

Valid	VPN	PPN
1	1	5

Virt

load 0x1000
load 0x1004
load 0x1008
load 0x100c

Phys

load 0x0004
load 0x5000
(TLB hit)
load 0x5004
(TLB hit)
load 0x5008
(TLB hit)
load 0x500C

PERFORMANCE OF TLB?

```
int sum = 0;
for (i=0; i<N; i++) {
    sum += a[i];
}
```

Would hit rate get better or worse
with smaller pages?

Miss rate of TLB: $\# \text{TLB misses} / \# \text{TLB lookups}$

Let $N = 2048$

$\# \text{TLB lookups?}$ number of accesses to $a = 2048$

$\# \text{TLB misses?}$

= number of unique pages accessed
= $2048 / (\text{elements of 'a' per 4K page})$
= $2K / (4K / \text{sizeof(int)}) = 2K / 1K$
= 2

Miss rate? $= 2/2048 < 0.1\%$

Hit rate? $(1 - \text{miss rate}) > 99.9\%$

TLB PERFORMANCE

How can system improve hit rate given fixed number of TLB entries?

Increase page size:

Fewer unique page translations needed to access same amount of memory

Called huge pages

On x86 normal pages are 4KB but huge pages can be 2MB or even 1GB!

Malloc gives normal 4KB pages

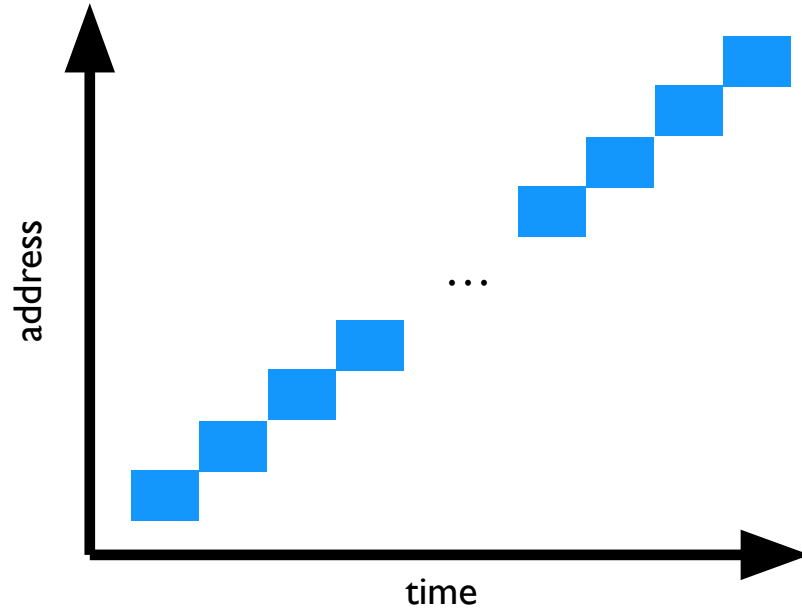
mmap(..., MAP_HUGETLB, ...) – explicitly asked by programmer

Sometimes, internally OS may decide to put data in a huge page

Types of Locality

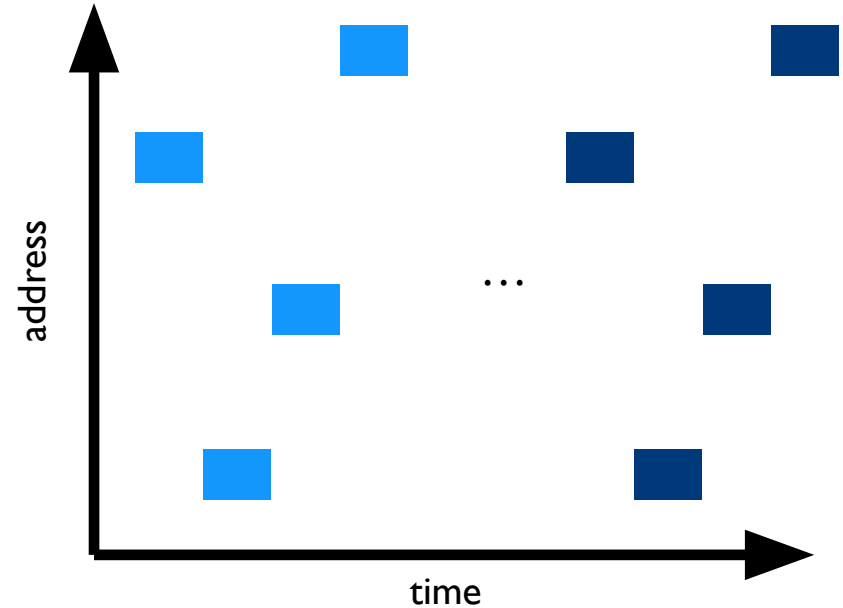
Spatial Locality

Sequential Accesses



Temporal Locality

Repeated Random Accesses



TLB CAN HELP WITH BOTH TYPES OF LOCALITY

Spatial Locality: future access to neighboring data

Temporal Locality: future access to the same data

Spatial:

- Access same page repeatedly; need same vpn -> ppn translation
- Same TLB entry re-used immediately

Temporal:

- Access same address near in future
- Same TLB entry re-used in near future
- How near in future? How many TLB entries are there?

HOW TO EVICT TLB ENTRIES

Common cache eviction policy (not just for TLB): LRU

Evict the least-recently used TLB slot

LRU TROUBLES



Valid	Virt	Phys
0	?	?
0	?	?
0	?	?
0	?	?

Workload repeatedly accesses same offset (0x01) across 5 pages (strided access), but only 4 TLB entries

What is the TLB miss rate?

Sometimes, random eviction policy can avoid such pathologies and thus better

CONTEXT SWITCHES

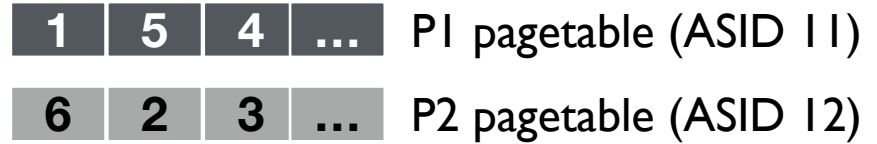
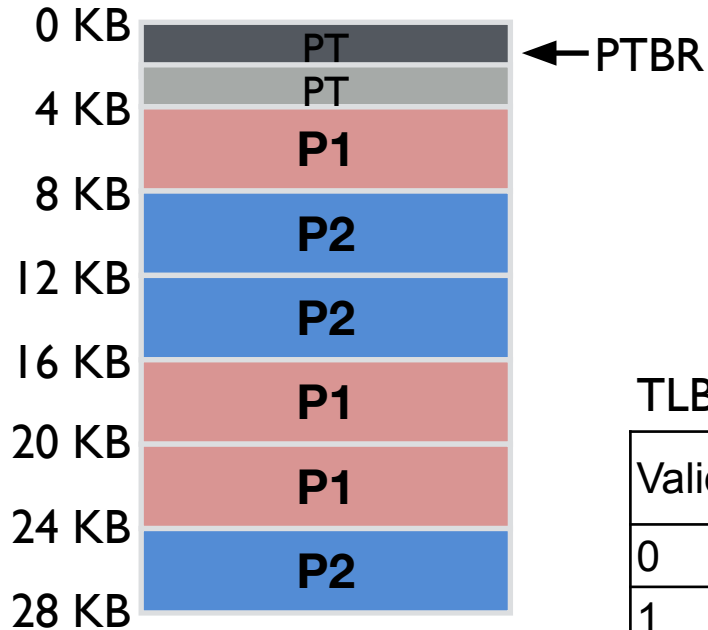
What happens if a process uses cached TLB entries from another process?

1. Flush entire TLB on each context switch (`void flush_tlb_all(void)` in Linux)

Lose all recently cached translations

2. Track which entries are for which process
 - Address Space Identifier
 - Tag each TLB entry with an 8-bit (or 16-bit) ASID
 - Must match ASID on lookups

TLB Example with ASID

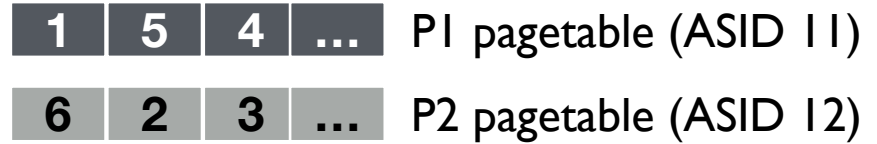
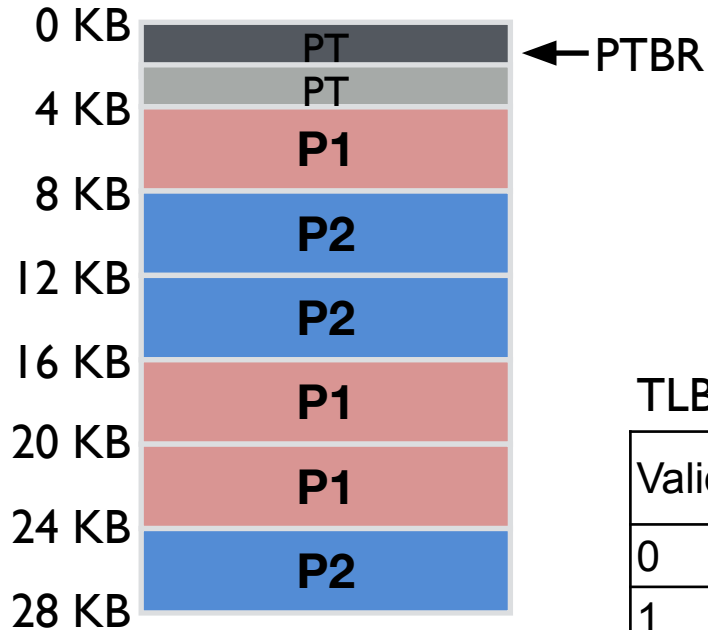


Virtual	Physical
load 0x1444 ASID: 12	
load 0x1444 ASID: 11	

TLB:

Valid	Virt	Phys	ASID
0	1	9	11
1	1	5	11
1	1	2	12
1	0	1	11

TLB Example with ASID



Virtual	Physical
load 0x1444 ASID: 12	load 0x2444
load 0x1444 ASID: 11	load 0x5444

TLB:

Valid	Virt	Phys	ASID
0	1	9	11
1	1	5	11
1	1	2	12
1	0	1	11

Aside: why are there two entries for VPN 1 in ASID 11?

UNMAPPING MEMORY

A process may unmap certain regions of virtual memory (e.g., `munmap`)

OS must ensure that translations for that unmapped region is removed from TLB

Several ways to do this:

1. Flush all tlb entries for a process – `void flush_tlb_mm(struct mm_struct *mm)`

Inefficient - but useful for fork + exec

2. Flush specific range: `void flush_tlb_range(vm_area_struct *vma, unsigned start, unsigned end)`

TLB Performance

Context switches are expensive

Even with ASID, other processes “pollute” TLB

Discard process A’s TLB entries for process B’s entries

A CPU architecture can have multiple TLBs

- A TLB for data, and a TLB for instructions
- A TLB for regular pages, and a TLB for “super pages”

WHAT HAPPENS ON A TLB MISS?

Load the PTE from memory

Insert into TLB

Retry the instruction (and the memory access)

If H/W handles TLB Miss

CPU must know location page table

- CR3 register on x86
- Page table format fixed and agreed upon between HW and OS
- H/W “walks” the page table and fills TLB

If OS handles TLB Miss: “software-managed TLB”

- CPU forces OS to (raise privilege level and) jump to a trap handler
- Handler interprets page tables and locates PTE, inserts into TLB
- Return from trap to retry the instruction

TLB Summary

Paging is great, but accessing page tables for every memory access is slow

Cache recent page translations in TLB

- MMU performs TLB lookup on every memory access

TLB performance depends strongly on workload

- Sequential workloads perform well
- Workloads with temporal locality can perform well

Context switches

- Flush TLB on every context switch
- Add ASID to every TLB entry

TLB miss handling: hardware or OS

NEXT CLASS

But, but, but...what about the fact that page tables are massive?